

Day05 Part C 学习文档 v1.0 : Tool Schema 设计 (Action Contract)

本文是《从零实现 Agent Runtime》学习阶段的 Day05 Part C 正式学习文档。

Day05 Part A 建立了 Tool Calling 的基础模型：LLM 生成 Tool Call Intent，Runtime 负责验证、执行和回写 Observation。Day05 Part B 进一步说明 Tool Decision 的本质是 Goal-driven Action Selection。Part C 继续追问：如果 LLM 要选择并使用 Tool，Runtime 应该如何描述这些 Tool，才能让模型稳定理解、选择和调用？

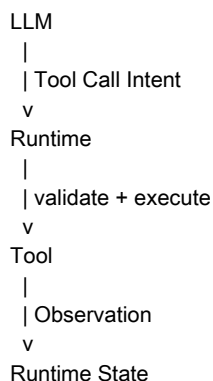
本节定位

Day05 的主题是 Tool Calling，也就是 Agent Runtime 的 Execution Engine。

Part A 回答：

Tool Calling 是什么？

核心是：



Part B 回答：

LLM 为什么会决定调用 Tool？

核心是：

Goal-driven Action Selection

Part C 回答：

Runtime 如何把可用能力描述给 LLM，让 LLM 能正确理解、选择、填参并解释结果？

核心是：

Tool Schema Design

本节的核心结论是：

Tool Schema 不是普通 API 文档，而是 LLM 与 Runtime 之间的行动契约。它定义模型可见的 Action Space、参数约束、结果语义和能力边界，因此直接影响 Tool Selection、参数生成、Observation 理解和后续规划。

目录

与 Part B 的联系

Tool Schema 是什么

Tool Schema 在 Agent Runtime 中的位置

Tool Schema 为什么是 Action Contract

Tool Schema 的三个基础字段

Tool Schema 是结构化 Prompt

Tool Schema 如何影响 Agent 行为

为什么不应该设计万能 Tool

Tool 粒度设计

工业 Agent 的 Tool 设计原则

Tool Schema 与 Context Engineering 的关系

Tool Schema 如何影响 Tool Decision

Tool Schema 优化

工业级 Tool Contract

Input Schema

Output Schema

Error Schema

Tool Metadata 与 Permission

Output Semantic Schema : LLM 如何理解返回枚举

OpenAI Function Calling / MCP 对应关系

mini-agent-runtime 中的最小数据模型

工业级实现 Notes

工业术语映射

面试视角

写书素材

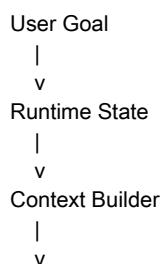
本 Part 核心认知升级

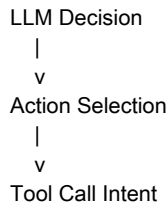
下一节学习计划

本章思考题

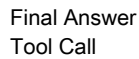
与 Part B 的联系

Part B 的核心链路是：





也就是说，LLM 每一轮不是只生成回答，而是在选择下一步行动：



但是，如果 LLM 要选择 Tool，它必须先知道：

Runtime 当前开放了哪些能力；

每个能力适合什么任务；

每个能力不适合什么任务；

调用时需要哪些参数；

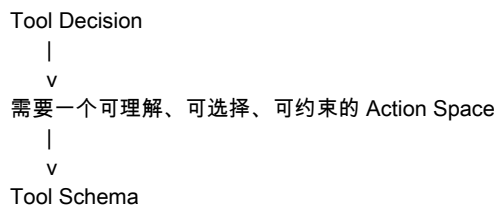
参数应该满足什么约束；

工具返回的结果代表什么；

工具失败时如何继续规划。

这些信息不是模型凭空知道的，而是 Runtime 通过 Tool Schema 提供给模型的。

因此 Part C 站在 Part B 的下一层：



Part B 研究的是：

如何选择 Action。

Part C 研究的是：

如何设计 Action Space。

Tool Schema 是什么

最直观的 Tool Schema 长这样：

```
{
  "name": "search_files",
  "description": "Search files in the workspace by keyword or regular expression.",
  "parameters": {
    "type": "object",
    "properties": {
      "query": {
        "type": "string",
        "description": "Keyword or regular expression to search for."
      },
      "path": {
        "type": "string",
        "description": "Directory scope for the search."
      }
    }
  }
}
```

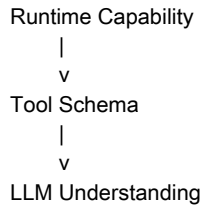
```
    },
    "required": ["query"]
  }
}
```

初看它像 API 定义。

但在 Agent Runtime 中，更准确的理解是：

Tool Schema 是 Runtime 提供给 LLM 的能力描述协议。

它让 LLM 理解：



Tool Schema 不是只告诉程序：

函数有哪些参数

而是告诉模型：

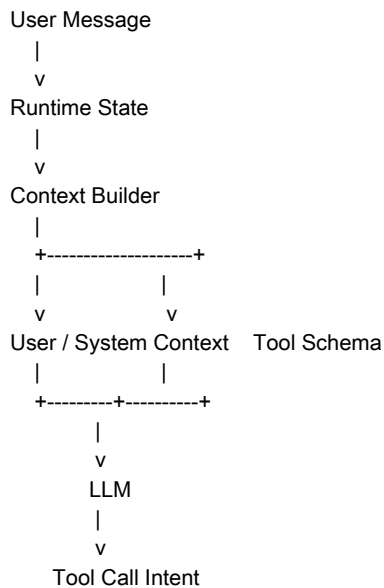
你現在有哪些行動能力
什麼情況下應該使用
什麼情況下不應該使用
參數應該如何構造
結果應該如何理解

所以 Tool Schema 的真正作用是：

让模型理解 Runtime 当前开放了哪些行动能力。

Tool Schema 在 Agent Runtime 中的位置

从 Runtime Loop 看：



Tool Schema 实际上也是 Context 的一部分。

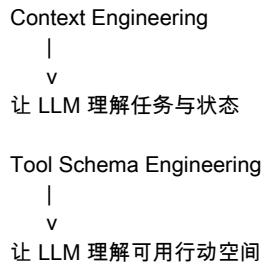
Day04 的核心思想是：

Context 决定 LLM 看什么。

Part C 增加了一层：

Tool Schema 决定 LLM 能做什么。

二者合在一起：

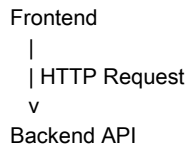


这就是为什么 Tool Schema 不只是后端接口定义，而是 Agent Runtime 的重要设计面。

Tool Schema 为什么是 Action Contract

普通程序之间靠 API Contract 协作。

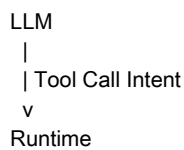
例如：



双方靠 API Contract 理解：

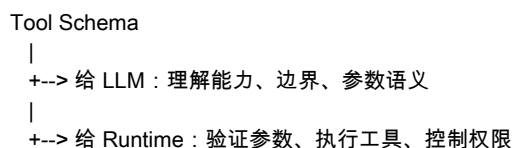
- endpoint 是什么；
- request body 怎么写；
- response 是什么；
- error 怎么处理。

Agent Runtime 中也存在类似契约：



但这里的调用者不是普通程序，而是 LLM。

因此 Tool Schema 需要同时面向两边：



它不是简单的 Function Signature，而是：

LLM 与 Runtime 的 Action Contract

这份契约描述的是：

- LLM 可以提出什么行动；
- Runtime 接受什么参数形状；

Runtime 如何验证和执行；
Tool Result 如何回流给 LLM；
失败、权限、风险如何表达。

Tool Schema 的三个基础字段

最基础的 Tool Schema 通常包含三个字段：

```
name
description
parameters
```

它们看起来简单，但分别影响模型的不同决策层。

name：能力名称

name 告诉模型：

这个能力叫什么。

差的命名：

```
{
  "name": "run"
}

{
  "name": "do_task"
}

{
  "name": "handle"
}
```

这些名字的问题是边界太模糊，模型很难从名称中判断能力用途。

更好的命名：

```
{
  "name": "search_files"
}

{
  "name": "read_file"
}

{
  "name": "query_order_status"
}
```

好的 Tool Name 往往采用：

动词 + 对象

例如：

- search_files
- read_file
- write_file
- query_order_status
- create_refund_request

- send_customer_email

命名本身会影响 Tool Selection。

如果两个工具分别叫：

```
search
search_files
```

模型更容易把文件查找任务匹配到 search_files，因为名称已经暴露了能力边界。

description：真正影响模型行为的字段

description 是 Tool Schema 中最接近 Prompt 的部分。

它不应该只写：

```
{
  "description": "Search files."
}
```

更好的写法是：

```
{
  "description": "Search files in the current workspace by keyword or regular expression. Use this when you need to locate code, configuration, documents, or other files."
}
```

好的 description 应该回答：

这个 Tool 做什么；

什么时候应该使用；

什么时候不应该使用；

它的输入输出语义是什么；

它的能力边界是什么。

可以写成：

```
Use this tool when ...
Do not use this tool when ...
This tool returns ...
```

例如：

```
{
  "name": "query_order_status",
  "description": "Query the current status of a customer order by order id. Use this when the user asks about shipping, delivery, order progress, or order status."
}
```

这个 description 同时给出：

- 正向触发条件；
- 负向边界；
- 业务语义；
- 与其他 Tool 的区别。

parameters：决定调用质量

parameters 决定 LLM 生成 Tool Call Arguments 的质量。

差的参数设计：

```
{
  "name": "query_order",
  "parameters": {
    "order_id": "string",
    "status": "string"
  }
}
```

```
"parameters": {  
  "input": "string"  
}
```

当用户说：

查询我的订单。

模型不知道 input 应该填什么：

```
{  
  "input": "查询我的订单"  
}
```

还是：

```
{  
  "input": "ORD-10001"  
}
```

更好的 Schema：

```
{  
  "name": "query_order_status",  
  "parameters": {  
    "type": "object",  
    "properties": {  
      "order_id": {  
        "type": "string",  
        "description": "The order id, such as ORD-10001."  
      }  
    },  
    "required": ["order_id"]  
  }  
}
```

这会让模型更稳定地生成：

```
{  
  "order_id": "ORD-10001"  
}
```

参数设计要避免：

- 使用过于抽象的 input、data、payload；
- 把多个语义挤进一个字符串；
- 让模型猜业务字段；
- required 字段不明确；
- 参数描述只写类型，不写语义。

Tool Schema 是结构化 Prompt

Prompt 会影响模型回答。

Tool Schema 会影响模型行动。

因此可以把 Tool Schema 理解为：

Structured Prompt for Action

普通 Prompt 描述：

你是谁？

你应该如何回答？
当前任务是什么？

Tool Schema 描述：

你能做什么？
何时做？
如何做？
做完后结果如何表达？

所以 Tool Schema Engineering 不是纯后端工程，而是 Prompt Engineering、Interface Design 和 Capability Design 的交叉点。

如果把 Schema 写得模糊，模型的行为也会模糊。

如果把 Schema 写得清晰，模型的行动就更可控。

Tool Schema 如何影响 Agent 行为

考虑用户请求：

帮我找一下项目里面用户登录相关代码。

模糊 Tool

```
{
  "name": "execute",
  "description": "Execute a command.",
  "parameters": {
    "type": "object",
    "properties": {
      "command": {
        "type": "string"
      }
    }
  }
}
```

这个 Tool 给模型的行动空间太大。

模型可能生成：

```
{
  "command": "find login"
}
```

也可能生成：

```
{
  "command": "grep -r login ."
}
```

甚至可能生成有风险的命令。

明确 Tool

```
[
  {
    "name": "search_files",
    "description": "Search files in the workspace by keyword or regular expression.",
    "parameters": {
      "type": "object",
      "properties": {
        "query": {
          "type": "string",
          "description": "Search keyword or regular expression."
        }
      }
    }
  }
]
```

```

    },
    "path": {
      "type": "string",
      "description": "Directory scope."
    }
  },
  "required": ["query"]
}
},
{
  "name": "read_file",
  "description": "Read the content of a specific file in the workspace.",
  "parameters": {
    "type": "object",
    "properties": {
      "path": {
        "type": "string",
        "description": "File path to read."
      }
    }
  },
  "required": ["path"]
}
}
]

```

这时模型更容易形成稳定行动序列：

```

search_files(query="login")
read_file(path="...")

```

Tool Schema 实际影响：

模型选择哪个 Tool；

模型是否调用 Tool；

模型生成什么参数；

Runtime 能否验证参数；

后续 Observation 能否被理解；

整个 Agent Loop 是否稳定。

为什么不应该设计万能 Tool

一个常见误区是设计万能 Tool：

```

{
  "name": "execute_task",
  "description": "Execute any task.",
  "parameters": {
    "type": "object",
    "properties": {
      "task": {
        "type": "string"
      }
    }
  },
  "required": ["task"]
}

```

它看起来能力很强，但对 Agent 来说很危险。

原因是 Action Space 太大：

execute_task
|
v
任何事情都可能被塞进 task

这会带来三个问题。

Decision Difficulty 增加

万能 Tool 无法帮助模型判断具体动作。

模型面对的是：

直接回答？
调用 execute_task？
task 里写什么？

由于边界太宽，模型更容易错误调用或不调用。

参数复杂

万能 Tool 通常会退化成自然语言参数：

```
{  
  "task": "帮用户查订单，然后如果没到就催一下物流"  
}
```

这等于把本该由 Runtime 管理的流程塞回给模型，让执行层不可控。

Runtime 无法控制

如果所有事情都通过一个 execute_task，Runtime 很难做：

- 权限判断；
- 风险分级；
- 审计记录；
- 超时控制；
- 人工审批；
- 失败恢复；
- 工具级别指标统计。

因此生产 Agent 更倾向于：

Capability Decomposition

也就是把能力拆成边界清晰的小 Tool。

Tool 粒度设计

Tool 粒度是工业 Agent 的核心设计问题。

可以把 Tool 设计放在一个光谱上：

过大粒度	过小粒度
v	v
万能 Tool	原子操作 Tool

目标不是越大越好，也不是越小越好，而是找到：

Optimal Capability Surface

大粒度 Tool 的问题

例如：

```
{  
  "name": "manage_order",  
  "description": "Manage customer orders."  
}
```

问题：

- 能力边界不清；
- Tool Selection 难；
- 参数容易变复杂；
- Runtime 权限控制困难；
- 返回结果难以结构化；
- 很难区分查询、取消、退款、催发货等业务动作。

过小粒度 Tool 的问题

例如：

```
click_button  
type_text  
press_enter  
wait  
read_pixel
```

这些 Tool 过于底层。

问题：

- Tool 数量过多；
- Action Space 过大；
- 模型需要自己规划大量低级动作；
- 一步错就会累积失败；
- 业务意图和 Tool 之间距离太远。

更好的粒度

企业客服场景中，较好的 Tool 往往是业务动作级：

```
query_order_status  
create_refund_request  
cancel_order  
update_shipping_address  
send_customer_notification
```

Coding Agent 场景中，较好的 Tool 往往是工程动作级：

```
search_files  
read_file  
edit_file  
run_tests  
inspect_git_diff
```

这些 Tool 既不是万能入口，也不是底层 UI 操作，而是人类完成任务时自然使用的动作。

工业 Agent 的 Tool 设计原则

Principle 1：单一职责

一个 Tool 应该对应一个清晰能力。

差：

```
handle_order
```

好：

```
query_order_status
cancel_order
create_refund_request
```

单一职责可以降低模型选择难度，也方便 Runtime 做权限和审计。

Principle 2：Tool 名称体现动作

名称最好体现：

verb + object

例如：

```
search_files
read_file
write_file
query_order_status
create_refund_request
```

模型会把名称作为决策线索。名称越贴近用户意图，Tool Selection 越稳定。

Principle 3：Tool 边界匹配业务能力

企业 Agent 不应该只暴露底层技术能力：

```
database_query
http_request
execute_sql
```

更好的做法是暴露业务能力：

```
query_order_status
lookup_customer_profile
create_refund_request
```

原因是用户目标通常是业务目标，不是技术动作。

业务动作级 Tool 更容易让 LLM 正确规划，也更安全。

Principle 4：Description 写出正负边界

好的 description 不只是说明 Tool 做什么，还应该说明：

```
Use this when ...
Do not use this when ...
```

例如：

```
{
  "name": "query_order_status",
  "description": "Query shipping and delivery status for an existing order. Use this when the user asks where an order is, whether it has shipped, or"
}
```

负向边界可以减少工具误用。

Principle 5 : 参数语义要明确

差：

```
{
  "properties": {
    "id": {
      "type": "string"
    }
  }
}
```

好：

```
{
  "properties": {
    "order_id": {
      "type": "string",
      "description": "The customer order id, such as ORD-10001."
    }
  }
}
```

参数名和 description 都应该提供业务语义。

Tool Schema 与 Context Engineering 的关系

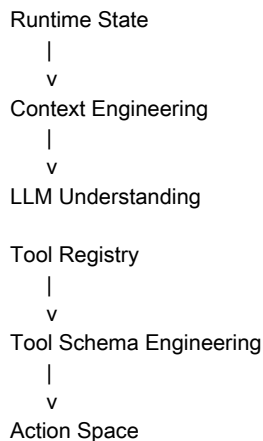
Day04 的 Context Engineering 解决：

LLM 应该看见什么？

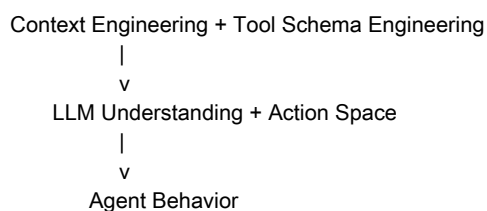
Tool Schema Engineering 解决：

LLM 可以做什么？

二者共同决定 Agent 行为：



合起来：



如果 Context 告诉模型用户目标，但 Tool Schema 没有表达好能力边界，Agent 仍然会行动不稳。

如果 Tool Schema 很清楚，但 Context 没有提供必要状态，Agent 也可能选错 Tool 或填错参数。

Tool Schema 如何影响 Tool Decision

Tool Schema 是 LLM 决策上下文的一部分。

当 Runtime 把多个 Tool Schema 放进请求时，模型看到的不是抽象函数列表，而是一个可选行动空间：

```
Tool A
Tool B
Tool C
Final Answer
```

模型会基于：

用户目标；

当前上下文；

Tool Name；

Tool Description；

Tool Parameters；

Runtime 指令；

模型自身能力；

来决定下一步行动。

因此修改 Tool Description 可能改变 Agent 行为。

例如：

```
{
  "name": "search",
  "description": "Search information."
}
```

和：

```
{
  "name": "search_files",
  "description": "Search local workspace files by keyword or regular expression. Use this before reading files when you need to locate relevant code."
}
```

在用户问：

找一下用户登录逻辑在哪里。

第二个 Schema 更容易让模型选择文件搜索，而不是直接回答或误用 Web 搜索。

这说明：

Tool Schema 是结构化决策提示。

它会改变模型在不同 Action 之间的概率分布。

Tool Schema 优化

Tool Schema 优化类似一种 Agent 行为调优。

不是改模型权重，而是改模型看到的 Action Space。

优化方向包括：

Description 增强

从：

```
{
  "description": "Query order."
}
```

改为：

```
{
  "description": "Query the shipping and delivery status of an existing customer order. Use this when the user asks where the order is, whether it has"
}
```

参数语义增强

从：

```
{
  "id": {
    "type": "string"
  }
}
```

改为：

```
{
  "order_id": {
    "type": "string",
    "description": "The order id from the user or current conversation, such as ORD-10001."
  }
}
```

减少歧义 Tool

如果有两个 Tool：

```
search
lookup
```

模型可能不知道区别。

应该改成：

```
search_files
lookup_customer_profile
```

或者在 description 中写清楚边界。

做 Tool Calling Evaluation

可以构造评测集：

```
用户问题：我的包裹什么时候到？
期望 Tool：query_order_status
期望参数：order_id
```

评测维度：

是否调用 Tool；

是否选择正确 Tool；

参数是否正确；

是否误调用；

Tool Result 后是否正确继续回答；

Error Result 后是否能恢复。

这就是：

Tool Calling Evaluation

工业级 Tool Contract

基础 Tool Schema 通常只有：

name
description
parameters

但生产级 Agent Tool 需要更完整的 Tool Contract：

Tool Contract
|
+-- name
+-- description
+-- inputSchema
+-- outputSchema
+-- errorSchema
+-- metadata
+-- permission

从 Function 到 Tool Contract 的升级是：

Function Definition
|
v
LLM 可调用接口
|
v
Tool Contract
|
v
LLM + Runtime + Policy + Observation 的完整契约

工业级 Tool 不只要解决：

怎么调用

还要解决：

谁能调用
何时调用
调用后返回什么
失败后怎么恢复
Runtime 如何控制
LLM 如何理解结果

Input Schema

Input Schema 定义 Tool Call Arguments 的结构。

例如：

```
{
  "inputSchema": {
    "type": "object",
    "properties": {
      "order_id": {
        "type": "string",
        "description": "The customer order id, such as ORD-10001."
      }
    }
  }
}
```

```

    }
  },
  "required": ["order_id"]
}

```

required 非常重要

如果没有 required :

```

{
  "properties": {
    "order_id": {
      "type": "string"
    }
  }
}

```

模型可能生成 :

```

{}

```

Runtime 需要额外处理缺参。

加上 required 后 :

```

{
  "required": ["order_id"]
}

```

模型会更明确地知道这个参数必须提供 , Runtime 也能更直接地验证。

参数描述不是给开发看的

参数 description 应该写给 LLM。

差 :

```

{
  "order_id": {
    "type": "string",
    "description": "id"
  }
}

```

好 :

```

{
  "order_id": {
    "type": "string",
    "description": "The order id mentioned by the user or stored in the current conversation state. It usually starts with ORD-."
  }
}

```

Output Schema

Output Schema 定义 Tool Result 的结构。

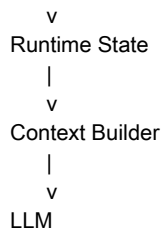
很多初学者只重视 Input Schema , 忽略 Output Schema。

但在 Agent Runtime 中 , Tool Result 会作为 Observation 回流 :

```

Tool Execution
|
v
Tool Result
|

```



因此 Output Schema 直接影响下一轮 LLM 如何理解世界。

例如：

```
{
  "outputSchema": {
    "type": "object",
    "properties": {
      "order_id": {
        "type": "string"
      },
      "status": {
        "type": "string",
        "enum": ["PENDING", "PAID", "SHIPPED", "DELIVERED", "CANCELLED"]
      },
      "delivery_date": {
        "type": "string",
        "description": "Expected delivery date in YYYY-MM-DD format."
      }
    },
    "required": ["order_id", "status"]
  }
}
```

Output Schema 的价值：

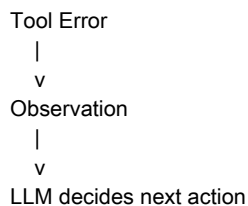
- 降低模型理解结果的成本；
- 让模型知道字段含义；
- 让 Runtime 可以验证 Tool Result；
- 让后续 Context Builder 可以选择性投影；
- 让 Evaluation 可以检查结果是否符合契约。

Error Schema

生产 Agent 必须设计错误。

普通程序遇到错误可以抛异常。

Agent 遇到错误后，还需要继续规划下一步：



因此错误也应该结构化。

例如：

```
{
  "errorSchema": {
```

```

"type": "object",
"properties": {
  "code": {
    "type": "string",
    "enum": ["ORDER_NOT_FOUND", "PERMISSION_DENIED", "TEMPORARY_UNAVAILABLE"]
  },
  "message": {
    "type": "string"
  },
  "recoverable": {
    "type": "boolean"
  },
  "suggested_action": {
    "type": "string",
    "description": "Recommended next step for the agent."
  }
},
"required": ["code", "message", "recoverable"]
}

```

错误结果可以是：

```

{
  "code": "ORDER_NOT_FOUND",
  "message": "No order was found for ORD-10001.",
  "recoverable": true,
  "suggested_action": "Ask the user to confirm the order id."
}

```

这样 LLM 就能继续：

请您确认一下订单号是否正确。

Error Schema 是 Agent 自恢复能力的重要基础。

Tool Metadata 与 Permission

Tool Schema 面向 LLM。

Tool Metadata 面向 Runtime。

二者应该分开。

给 LLM 的信息

```

name
description
inputSchema
outputSchema summary

```

这些信息帮助模型理解能力和参数。

给 Runtime 的信息

```

riskLevel
timeoutMs
requiresApproval
permissionScope
sideEffect
idempotent

```

这些信息帮助 Runtime 控制执行。

例如：

```
type ToolDefinition = {
  name: string;
  description: string;
  inputSchema: JSONSchema;
  outputSchema?: JSONSchema;
  errorSchema?: JSONSchema;
  metadata: {
    riskLevel: "low" | "medium" | "high";
    timeoutMs: number;
    requiresApproval: boolean;
    sideEffect: "none" | "read" | "write" | "external";
  };
};
```

为什么要分开？

因为：

Tool Schema 服务于 LLM 理解能力边界。
Runtime Metadata 服务于执行控制。

例如删除文件：

```
delete_file
```

LLM 需要知道这个 Tool 用于删除文件。

Runtime 还需要知道它是高风险、有副作用、需要人工审批。

这会连接到后续 Day05 Part F：Permission & Human Approval。

Output Semantic Schema：LLM 如何理解返回枚举

学习过程中有一个关键追问：

Tool 返回 "status": "SHIPPED" 时，LLM 怎么知道 SHIPPED 是什么意思？需要代码写好枚举值吗？

答案是：

需要设计，但不一定只是传统代码 enum。LLM
不理解你的业务枚举，它理解的是你提供给它的语义上下文。

最简单情况：只返回枚举值

Tool 返回：

```
{
  "order_id": "ORD-10001",
  "status": "SHIPPED",
  "delivery_date": "2026-07-30"
}
```

模型可能凭通用知识猜到 SHIPPED 表示已发货。

但生产系统不应该依赖模型猜。

因为业务枚举可能是：

```
RTO
RTS
POD_PENDING
PARTIALLY_FULFILLED
MANUAL_REVIEW_REQUIRED
```

这些枚举对模型并不一定有稳定语义。

方案 A：返回语义化字段

可以让 Tool Result 直接包含面向 Agent 的语义字段：

```
{
  "order_id": "ORD-10001",
  "status": "SHIPPED",
  "status_label": "已发货",
  "status_description": "订单已从仓库发出，正在运输途中。",
  "delivery_date": "2026-07-30"
}
```

这样 LLM 不需要猜枚举。

它可以直接理解：

已发货，预计 2026-07-30 送达。

这体现了一个核心原则：

Tool Result 也是 Context。

方案 B：Output Schema 定义枚举

可以在 Output Schema 中定义枚举：

```
{
  "outputSchema": {
    "type": "object",
    "properties": {
      "status": {
        "type": "string",
        "enum": ["PENDING", "PAID", "SHIPPED", "DELIVERED", "CANCELLED"],
        "description": "Current order status. SHIPPED means the order has left the warehouse and is in transit."
      }
    }
  }
}
```

这对 Runtime 和 LLM 都有价值。

对 Runtime：

- 可以验证返回值；
- 可以发现异常枚举；
- 可以保持接口契约稳定。

对 LLM：

- 可以理解枚举语义；
- 可以减少误解释；
- 可以生成更准确的用户回答。

方案 C：Enum + Description Mapping

更完整的做法是给出映射：

```
{
  "status": "SHIPPED",
  "status_meaning": {
    "code": "SHIPPED",
    "label": "已发货",
    "description": "订单已经交给物流承运商，正在运输途中。",
    "user_message_hint": "您的订单已发货，预计会在 delivery_date 前后送达。"
  }
}
```

```
}  
}
```

这就是：

Semantic Schema

结构 Schema 只告诉模型：

status 是 string

语义 Schema 告诉模型：

status 的业务含义是什么
应该如何向用户解释

方案 D：直接返回面向 Agent 的结果

还有一种工业做法是 Tool 直接返回 Agent-friendly Result：

```
{  
  "order_id": "ORD-10001",  
  "status": {  
    "code": "SHIPPED",  
    "label": "已发货",  
    "description": "订单已发货，正在运输途中。"  
  },  
  "next_best_action": "Tell the user the package is in transit and provide the delivery date.",  
  "user_visible_summary": "您的订单已发货，预计 2026-07-30 送达。"  
}
```

这不是普通 API 风格，而是：

Agent-oriented API Design

也就是：

为 Agent 设计 API，而不是只为程序设计 API。

代码里需要 enum 吗

通常需要。

在代码里，枚举可以用于：

类型约束；

服务端校验；

防止非法状态；

生成文档；

生成 Output Schema；

前端展示映射；

Agent 语义解释。

但只在代码里写 enum 还不够。

因为 LLM 看不到或不理解代码里的 enum。

需要把 enum 的语义通过以下方式暴露给 Agent：

Output Schema
Semantic Mapping
Tool Result Fields
System Prompt / Domain Context

这正好补齐 Part C 的一个关键点：

Tool Schema 不只是 Input Schema，还包括 Output Schema 和 Semantic Schema。

OpenAI Function Calling / MCP 对应关系

OpenAI Function Calling

OpenAI Function Calling 中常见结构：

```
{
  "type": "function",
  "function": {
    "name": "query_order_status",
    "description": "Query shipping and delivery status for an order.",
    "parameters": {
      "type": "object",
      "properties": {
        "order_id": {
          "type": "string"
        }
      }
    },
    "required": ["order_id"]
  }
}
```

这对应基础 Tool Schema：

```
name
description
parameters
```

MCP

MCP 更强调 Tool Discovery。

Tool 可能长这样：

```
{
  "name": "query_order_status",
  "description": "Query shipping and delivery status for an order.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "order_id": {
        "type": "string"
      }
    }
  },
  "required": ["order_id"]
}
```

MCP 的核心价值是：

标准化 Agent Capability Discovery

也就是让 Agent Runtime 能从外部 server 发现可用 Tool，并把这些能力暴露给 LLM。

mini-agent-runtime 中的最小数据模型

在 mini-agent-runtime 中，可以先设计最小 Tool Schema：

```
type JSONSchema = {
```



```

type: string;
properties?: Record<string, JSONSchema>;
required?: string[];
description?: string;
enum?: string[];
items?: JSONSchema;
};

```

```

type ToolDefinition = {
  name: string;
  description: string;
  inputSchema: JSONSchema;
};

```

随着 Runtime 进入工业化，可以扩展为：

```

type ToolSideEffect = "none" | "read" | "write" | "external";
type ToolRiskLevel = "low" | "medium" | "high";

```

```

type ToolDefinition = {
  name: string;
  description: string;
  inputSchema: JSONSchema;
  outputSchema?: JSONSchema;
  errorSchema?: JSONSchema;
  metadata: {
    sideEffect: ToolSideEffect;
    riskLevel: ToolRiskLevel;
    timeoutMs: number;
    requiresApproval: boolean;
  };
};

```

Tool Call Intent 可以是：

```

type ToolCallIntent = {
  id: string;
  toolName: string;
  arguments: unknown;
};

```

Tool Result 可以是：

```

type ToolResult =
  | {
    ok: true;
    toolName: string;
    callId: string;
    output: unknown;
  }
  | {
    ok: false;
    toolName: string;
    callId: string;
    error: {
      code: string;
      message: string;
      recoverable: boolean;
      suggestedAction?: string;
    };
  };

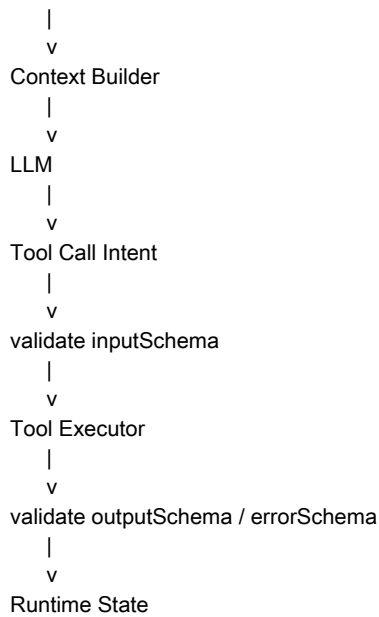
```

最小 Runtime 链路：

```

Tool Registry
|
v
select Tool Definitions

```



工业级实现 Notes

1. Tool Schema 需要版本管理

生产中的 Tool Schema 会变化。

例如：

```
query_order_status v1
- order_id

query_order_status v2
- order_id
- include_tracking_history
```

如果 Tool Schema 变化不受控，可能导致：

- 旧 Prompt 失效；
- 旧评测失效；
- 模型生成旧参数；
- Runtime 验证失败；
- Agent 行为突然变化。

因此 Tool Definition 应该包含版本：

```
type ToolDefinition = {
  name: string;
  version: string;
  description: string;
  inputSchema: JSONSchema;
};
```

2. Tool Schema 需要测试

Tool Schema 不能只靠人工感觉。

应该用 Evaluation 检查：

User Input -> Expected Tool -> Expected Arguments

例如：

```
{
  "input": "我的包裹什么时候到？",
  "expectedTool": "query_order_status",
  "expectedArguments": {
    "order_id": "ORD-10001"
  }
}
```

测试内容：

Tool Selection 是否正确；

参数是否完整；

参数类型是否正确；

是否误调用高风险工具；

Tool Result 后回答是否正确；

Error Result 后是否能恢复。

3. Tool Schema 需要按场景裁剪

不要把所有 Tool Schema 一次性塞给 LLM。

如果 Runtime 有 100 个 Tool，全部暴露会导致：

Action Space 爆炸

后续 Part D 的 Tool Registry 会解决：

Runtime 如何管理大量 Tool？

当前任务应该暴露哪些 Tool？

Tool Routing 如何降低 Action Space？

4. Tool Schema 需要与权限系统结合

有些 Tool 是只读的：

search_files
query_order_status

有些 Tool 会产生副作用：

write_file
cancel_order
create_refund_request
send_email

Tool Schema 描述能力，但是否执行必须由 Runtime Policy 决定。

这会连接到后续：

Permission & Human Approval

工业术语映射

| 本项目概念 | 工业术语 || --- | --- || Tool Schema | Function Schema / Tool Definition || Tool Contract | Tool Interface Contract || 能力描述 | Capability Description || 参数定义 | Input Schema / JSON Schema || 输出定义 | Output Schema || 错误定义 | Error Schema || 语义定义 | Semantic Schema || 行动空间 | Action Space || 能力拆分 | Capability Decomposition || 工具边界 | Capability Boundary || 工具粒度 | Tool Granularity || Tool 优化 | Tool Schema Optimization || Agent 友好 API | Agent-oriented API Design || 能力发现 | Capability

面试视角

Q1：为什么 Tool Schema 很重要？

回答：

Tool Schema 是 LLM 和 Runtime 之间的能力契约。它不仅描述工具接口，还定义模型可见的行动空间、参数约束和结果语义，直接影响 Tool Selection、参数生成和后续规划。

Q2：Tool Schema 和普通 API Schema 有什么区别？

回答：

API Schema 主要面向程序调用者，关注接口结构和类型；Tool Schema 面向 LLM 决策，除了接口结构，还需要描述什么时候使用、什么时候不要使用、能力边界、输出语义和错误恢复信息。

Q3：为什么一个万能 Tool 是坏设计？

回答：

万能 Tool 会让 Action Space 过大，模型难以稳定选择和填参，同时 Runtime 难以做权限控制、审计、风险分级和失败恢复。工业 Agent 更倾向于拆分职责清晰的业务动作级 Tool。

Q4：为什么修改 Tool Description 可以改变 Agent 行为？

回答：

Tool Schema 是 LLM Context 的一部分，Description 会影响模型对可用能力和适用场景的理解，从而改变 Tool Selection 的概率分布。优化 Description 本质是在优化模型的行动决策空间。

Q5：为什么 Tool 需要 Output Schema？

回答：

Tool Result 会作为 Observation 回流 Runtime State，并进入下一轮 Context。如果输出结构和语义不清，模型需要额外猜测结果含义，会降低 Agent 稳定性。Output Schema 能帮助模型理解结果，也方便 Runtime 验证和投影。

Q6：为什么 Agent Tool 需要 Error Schema？

回答：

Agent 遇到错误后不是简单抛异常，而是要继续规划下一步行动。结构化 Error Schema 可以告诉模型错误类型、是否可恢复以及建议动作，从而支持自恢复和更自然的用户交互。

Q7：status: "SHIPPED" 这种枚举需要怎么让 LLM 理解？

回答：

不应该依赖模型猜业务枚举。可以在 Output Schema 中定义 enum 和 description，也可以在 Tool Result 中返回 status_label、status_description、user_visible_summary 等语义字段。代码里的 enum 负责类型和校验，暴露给 Agent 的 Semantic Schema 负责模型理解。

Q8：Tool Schema 和 Runtime Metadata 为什么要分开？

回答：

Tool Schema 服务于 LLM 理解能力边界，Runtime Metadata 服务于执行控制，例如权限、风险、超时、人工审批和副作用。二者关注点不同，分开后更容易同时支持模型决策和 Runtime 安全控制。

写书素材

适合放进书里的英文总结：

A tool schema is not just a function signature. It is an action contract between the model and the runtime, shaping what the model can see, choose, call, and understand after execution.

中文版本：

Tool Schema 不只是函数签名，而是模型与 Runtime 之间的行动契约。它定义模型能看到什么能力、如何选择能力、如何调用能力，以及如何理解执行后的世界变化。

本 Part 可以使用的章节标题：

Tool Schema: Designing the Agent's Action Space

或者：

Tool Schema Is the Action Contract Between LLM and Runtime

本 Part 核心认知升级

完成 Part C 后，认知应从：

Tool Schema = 函数参数定义

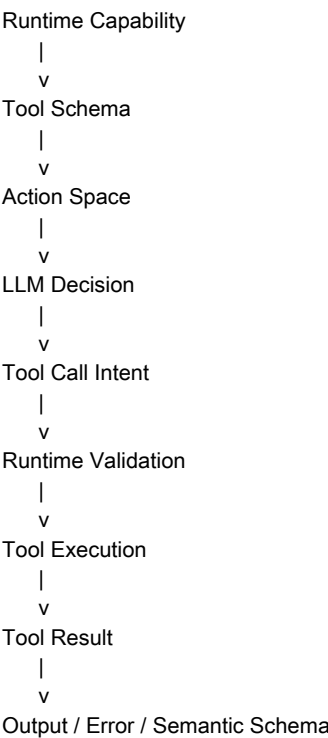
升级为：

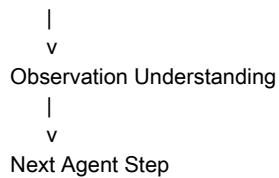
Tool Schema = Agent 的行动空间设计

再进一步：

Tool Schema = LLM 与 Runtime 之间的 Action Contract

完整链路：





Part C 的最终结论：

Agent 能力提升不只是换更大的模型，而是设计更好的 Tool Schema，让模型拥有更清晰、更稳定、更可控的行动空间。

下一节学习计划

Day05 Part D：Tool Registry。

Part C 解决的是：

单个 Tool 应该如何设计？

Part D 继续解决：

Runtime 如何管理大量 Tool？

重点问题：

Tool Registry 是什么；

Runtime 如何注册 Tool；

Tool Definition 和 Tool Implementation 如何分离；

当前任务应该暴露哪些 Tool；

Tool Routing 如何降低 Action Space；

动态 Tool Discovery 如何工作；

MCP 与 Tool Registry 的关系；

mini-agent-runtime 中如何实现最小 Tool Registry。

本章思考题

为什么 Tool Schema 不是普通 API 文档？

Tool Schema 为什么可以看作结构化 Prompt？

Tool Name 为什么会影响 Tool Selection？

好的 Tool Description 应该包含哪些信息？

为什么参数名不应该写成 input 或 payload？

为什么一个万能 Tool 会降低 Agent 稳定性？

Tool 粒度过大和过小分别有什么问题？

为什么企业 Agent 更适合暴露业务动作级 Tool？

Tool Schema 和 Context Engineering 有什么关系？

为什么修改 Tool Description 可能改变 Agent 行为？

为什么增加 Tool 可能反而降低 Agent 效果？

Input Schema、Output Schema、Error Schema 分别解决什么问题？

为什么 Tool Result 也是 Context？

LLM 如何理解 SHIPPED 这类业务枚举？

代码里的 enum 和 Agent 可见的 Semantic Schema 有什么区别？

Tool Schema 为什么需要版本管理？

如何设计 Tool Calling Evaluation？

Tool Schema 和 Runtime Metadata 为什么要分开？

Tool Schema 如何连接 Permission & Human Approval？

Tool Registry 为什么是 Part D 的自然下一步？

Source

- ChatGPT 分享学习记录：<https://chatgpt.com/share/6a66c34d-978c-83ee-a516-2fc9abb9446f>

- 本地源记录：[source/day05-part-c-chatgpt-share-source.md](#)